

This is exactly what was happening under the umbrella of the glNext project.

Also revealed was the fact that while Vulkan was to be the successor to OpenGL, it does not share its legacy. In fact Vulkan is to a large extent based on AMD's Mantle graphics API, which AMD graciously contributed to the Khronos Group. This is rather a generous move of AMD given how it will now become a vendor-neutral and platform-neutral framework under Khronos, thus eliminating any competitive advantage they had through a vendor-locked framework. To be honest though Mantle was unlikely to succeed as an AMD-only Windows-only technology, especially given the similar goals of DirectX 12.

So while Vulkan is a successor to OpenGL, it is also the successor to AMD Mantle. Given the existence of Vulkan, Mantle will no longer exist in its current form since it has essentially been superseded. The landscape of graphics API has suddenly cleared a lot, it is now down to three, Metal by Apple, DirectX 12 by Microsoft, and Vulkan by the Khronos Group.

Unlike Microsoft and Apple, the Khronos Group is a non-profit consortium that is focused entirely on creating open graphics standards. Apple and Microsoft are member of Khronos as well though among other major game development and hardware companies such as AMD, Intel, NVIDIA, Valve, Unity, EPIC Games, Blizzard, EA, Google, Mozilla, Oxide, PIXAR, and Oculus VR and many others.

If you think it odd that a new major standard API is based on a technology created by a single corporation for its own use, perish the thought. This is exactly how OpenGL started in the first place. OpenGL was a successor to IRISGL (Integrated Raster Imaging System Graphics Library), which was a creation of Silicon Graphics (SGI) designed for their own workstations. It was eventually open sourced and standardised as OpenGL.

What Vulkan Brings to the Table

One of the biggest advantage that Vulkan has over other graphics APIs is the fact that it is designed to work on all platforms, desktop or mobile, and operating systems. It can work with NVIDIA hardware, AMD hardware, and Intel hardware, and other graphics hardware available on mobile platforms such as Adreno.

This, of course, is something OpenGL already does, and very important for Vulkan being a fresh API. Where Vulkan is intended to excel, is where some of the biggest drawbacks lie in OpenGL currently. It is designed to take advantage of the current generations' architectures, and trends

*pointers

>> Interesting developer videos



*Python for Humans

A great talk by the creator of the popular Requests library for Python about good API design.

<http://dgit.in/PyForHumans>



*glNext: The Future of High Performance Graphics (Presented by Valve)

This GDC talk presents the new Vulkan API for the first time with talks by various people involved in its development.

<http://www.gdcvault.com/play/1022018/>



*Wearable Electronics with Becky Stern

If you are interested in wearable electronics, especially the DIY-ing, this great live video series by Adafruit Industries is one to watch.

<http://dgit.in/WearElec-Becky>



*Docker Tutorials

The Docker team has started a new series of video tutorials that showcase different aspects of Docker at a very basic introductory level.

<http://dgit.in/DockerTut>



in hardware architectures that are here to stay. When OpenGL was designed, both CPUs and GPUs were single-core affairs.

The trend these days is towards higher core counts at lower speeds. Octa-core mobile processors are increasingly common and are getting cheaper. Desktop and laptop core counts are also on the rise. We've already mentioned how the number of GPU cores had been on the rise, and is now in the territory of 16 physical cores and hundreds or even thousands of virtual cores.

Multi-threaded design

These trends do not play well with the way OpenGL is designed. While OpenGL-based games and applications can take advantage of multiple cores, all OpenGL interactions only run in a single thread. What this means is that even if the rest of the application code is spread across multiple processors, the speed of a single core defines how fast OpenGL can process graphics commands and pass them to the GPU.

Vulkan is explicitly designed to take advantage of multiple cores. It supports receiving commands from multiple threads at the same time allowing for better utilisation of multi-core hardware.

This is a design that has been proven to show performance gains since EA's Frostbite 3 engine (which powers all their modern games such as the newest Dragon Age: Inquisition and Battlefield 4) was ported to use Mantle, and showed real world improvements in performance to as much as 45%.

Lower driver overhead

Current graphics drivers do a lot, too much if you ask the developers of Vulkan. One of the goals of Vulkan then is to reduce how much management OpenGL does, and leave all that for the engine developer to handle.

Vulkan gives developers explicit control over the GPU, and removes everything but the most necessary components between the GPU and the software using it.

This increase in power, also brings with an increase in responsibility. Low-level features such as memory management, error validation and CPU-GPU synchronization are now the responsibility of the developer.

Vulkan even removes run-time error checking that was done by OpenGL. The idea behind this being that all the testing should be done beforehand, so error-checking need not run in production code. Also, despite error checking, GPU crashes are known to happen, so why pay with the overhead?